amU Faculté des sciences Aix Marseille Université

**Licence de maths - EADS**
**L2-L3 Maths Info**
**Modélisation, semestre 4**
**2025-2026**

---

**dcc 1**

# Débruitage de sons

Nous considérons dans ce problème la question du débruitage de deux signaux sonores.

Le débruitage est l'opération qui consiste à restaurer des données qui ont subi une perturbation lors de leur acquisition ou lors d'une transmission, ou à un autre moment de la chaine de traitement.

Quand on doit débruiter des données on a donc affaire à des observations $y$ d'un signal d'origine $u$ qui a subi une perturbation. Un cas classique est le cas du bruit additif indépendant du signal. Il est utilisé pour modéliser les bruits dûs aux dispositifs électroniques dans les appareils de mesure, le bruit d'agitation thermique et aussi les perturbations lors de transmissions (dans des canaux terrestres, ou par satellite)... On modélise le signal $y \in \mathbb{R}^N$ observé comme

$$y = u + \beta \tag{1}$$

avec $u \in \mathbb{R}^N$ le signal original non perturbé et $\beta \in \mathbb{R}^N$ une perturbation indépendante du signal.

Notre objectif est de restaurer deux signaux sonores bruités.

Les données sont à télécharger sur Ametice. Des routines Python pour vous aider dans les parties numériques sont disponibles dans la section 4.

## 1 Introduction à l'acoustique : notion de fréquence sonore

Nos signaux sont des signaux musicaux et à ce titre des signaux composés de notes jouées par des instruments de musique.

La branche de la physique qui s'intéresse aux phénomènes sonores s'appelle l'acoustique et nous allons ici en donner quelques concepts de base. Un son existe dès que la pression dans un gaz ou un liquide. Ainsi, le son physique perçu par un récepteur (que ce soit un microphone ou l'oreille humaine) peut être représenté par la fonction $t \mapsto P(t)$ décrivant la pression de l'air au récepteur à l'instant $t$. Cette fonction est obtenue à partir d'un phénomène qui a en général eu lieu à un autre endroit que le récepteur et qui a créé des mouvements d'air qui se sont propagés jusqu'au récepteur à l'aide de ce qu'on appelle les ondes acoustiques.

Cette vision de l'acoustique nous permet d'introduire la notion de note pure qui est n'importe quelle fonction G de la forme $t \mapsto A\sin(\omega t + \phi)$ où $A$ et $\phi$ sont des réels et $\omega$ un réel strictement positif. Le paramètre $A$ décrit ce qu'on appelle l'amplitude de la note et qui est attachée à la notion courante de volume sonore (plus $|A|$ est grand plus le son est fort dans le langage courant). Le paramètre $\phi$ est ce qu'on appelle le déphasage de la note pure et correspond à l'intensité du son à l'instant 0. Le paramètre $\omega$ s'appelle la pulsation et son unité dans le système international est le « par seconde »autrement appelé le Hertz. C'est la grandeur mathématique la plus naturelle pour représenter une note pure. Nous sommes plutôt habitués à manipuler, dans la vie courante, un autre concept, celui de fréquence. On remarque qu'une note pure est

décrite par une fonction périodique dont la période minimale est $T = 2\pi/\omega$ puisque la fonction sin est elle-même de période minimale $2\pi$. La fréquence $f$ de la note est définie comme l'inverse de la période et donc est liée à la pulsation par la formule $f = \omega/(2\pi)$. Puisque la pulsation et la fréquence ne diffèrent que d'un facteur sans dimension, l'unité de la fréquence est également le Hertz. La fréquence est la notion la plus naturelle d'un point de vue physique car une fréquence de 1Hz signifie que la note pure fera 1 oscillation complète par seconde.

**Question 1**

Bien que physiquement un son soit l'évolution dans le temps d'une quantité représentée par un réel (la pression) il est très courant en acoustique de manipuler des fonctions à valeurs complexes car elles simplifient très souvent la théorie et les calculs. Ici $A$, et $\phi$ sont des constantes réelles fixées. En particulier les fonctions du type $\mathbf{E}_\omega : t \mapsto e^{i\omega t}$ pour $\omega \in \mathbb{R}$ fixé permettent parfois de simplifier les écritures de fonctions plus élaborées.

*En physique les fréquences négatives n'existent pas mais celles-ci apparaissent dans nos calculs pour simplifier l'écriture !*

1. On considère $G$ une note pure définie sur $\mathbb{R}$. Dans cette question et seulement dans celle-là on prend $A = 1$, $\phi = \frac{\pi}{4}$. Pour les valeurs $\omega = 1, 4, 8$ tracer la partie réelle et la partie imaginaire de la fonction $\mathbf{E}_\omega$, puis la fonction $G$ sur $[-3\pi, 3\pi[$. Mêmes questions pour $\omega = 2\pi, 4\pi$ sur un intervalle choisi adéquatement.
2. Montrer qu'il existe $c \in \mathbb{C}$ (que l'on explicitera) tel que pour tout $t \in \mathbb{R}$ $G(t) = c\mathbf{E}_\omega(t) + \overline{c}\mathbf{E}_{-\omega}(t)$.
3. Soit $G_1 : t \mapsto A_1 \cos(\omega t + \phi_1) + A_2 \sin(\omega t + \phi_2)$ avec $A_1, A_2, \phi_1, \phi_2 \in \mathbb{R}$. Montrer qu'il existe $c_1 \in \mathbb{C}$ (que l'on explicitera) tel que pour tout $t \in \mathbb{R}$ $G_1(t) = c_1\mathbf{E}_\omega(t) + \overline{c_1}\mathbf{E}_{-\omega}(t)$.
4. Soit $G_2 : t \mapsto A\cos(\omega t + \phi_1)\sin(\omega t + \phi_2)$. Montrer qu'il existe $\omega_2 \in \mathbb{R}$ et $c_2, d_2 \in \mathbb{C}$ tels que pour tout $t \in \mathbb{R}$ $G_2(t) = c_2\mathbf{E}_{\omega_2}(t) + \overline{c_2}\mathbf{E}_{-\omega_2}(t) + d_2 + \overline{d_2}$. La fonction $G_2$ est-elle périodique ? Peut-on définir sa fréquence ?
5. Soit $G_3 : t \mapsto A\cos(\omega_1 t + \phi_1)\sin(\omega_2 t + \phi_2)$ avec $0 < \omega_1 < \omega_2$. Décomposer $G_3$ de la manière la plus simple possible à l'aide de fonctions du type $\mathbf{E}_\omega$.
6. On prend dans cette question $\omega_1 = \sqrt{2}$ et $\omega_2 = \sqrt{3}$. $G_3$ est-elle une fonction périodique ? Est-il possible de parler de « la fréquence »de $G_3$ ? Et si non « des fréquences »qui interviennent dans $G_3$ ? Détailler votre réponse.
7. Un instrument de musique joue toujours une note pure $t \mapsto \sin(2\pi\omega_0 t)$ ($\omega_0 \in \mathbb{R}^*$) auxquelles s'ajoutent des harmoniques c'est à dire des notes dont la fréquence est proportionnelle à la fréquence fondamentale $\omega_0$. On suppose ici que ces harmoniques s'écrivent $a^n \sin(n2\pi\omega_0 t)$ pour $n = 1, \ldots, N_0$ avec $|a| < 1$. Donner la fonction $t \mapsto F(t)$ mesurée aux bornes d'un haut-parleur et son allure pour $\omega_0$ et $N_0 \geq 4$ fixés que vous choisirez.

Pour l'être humain le système auditif (l'oreille et la partie du cerveau qui traite l'information qui en vient) est en fait vraiment sensible à la fréquence des sons (ou à leur pulsation) donc plus qu'à la valeur temporelle de la pression reçue. En effet si nous entendons deux sons purs $G_1 : t \mapsto A_1 sin(\omega_1 t)$ et $G_2 : t \mapsto A_2 sin(\omega_2 t)$, nous serons quasi instantanément capables de déterminer si $\omega_1 < \omega_2$ ou l'inverse : la plus grande valeur de fréquence sera associée au son le plus aigu alors que la plus petite valeur sera associée au son le plus grave. L'oreille humaine est capable de distinguer les sons dont la fréquence se situe entre $20Hz$ pour les sons les plus graves et $20kHz = 20000$ pour les sons les plus aigus. La fréquence est tellement importante dans la perception humaine qu'il a été décidé de donner conventionnellement des noms à certaines fréquences : par exemple la fréquence de $440Hz$ s'appelle le *la* (ou même le *la* de concert), $660Hz$ s'appelle *mi* (ou plus précisément *mi* pythagoricien), etc...

## 2 Numérisation des sons

La modélisation du son par une fonction du temps est ce qu'on appelle sa représentation analogique. Pour pouvoir agir efficacement sur le signal sonore à l'aide des outils de l'informatique moderne, il est nécessaire de transformer le signal analogique en signal numérique. En effet, un ordinateur ne pouvant manipuler que des données en quantité finie, la notion de fonction est inopérante. On fera l'hypothèse naturelle que les sons que l'on va manipuler ont une durée finie $T$. Le son est représenté par le signal analogique $G : [0, T] \mapsto \mathbb{R}$, qui est tout simplement une fonction de la variable réelle définie sur l'intervalle $[0, T]$. On peut ensuite le numérise par la procédure suivante dite d'échantillonnage.

On choisit $f_s > 0$ qui est la « cadence d'échantillonnage »ou encore « fréquence d'échantillonnage ». Le signal numérisé est obtenu en mesurant pour $0 \leq n \leq N-1$ $u_n = G\left(\frac{n}{f_s}\right) = G(nt_s)$. Le vecteur $u = (u_0, \dots u_{N-1}) = \left(G\left(\frac{0}{f_s}\right), \dots, G\left(\frac{(N-1)}{f_s}\right)\right)$ correspond donc à la numérisation du signal $G$ (voir figure 1) [1].

*On se rend compte sur cette figure 1 que si la cadence d'échantillonnage $f_s$ est trop petite, le signal pourrait être dégradé au moment de la numérisation.*

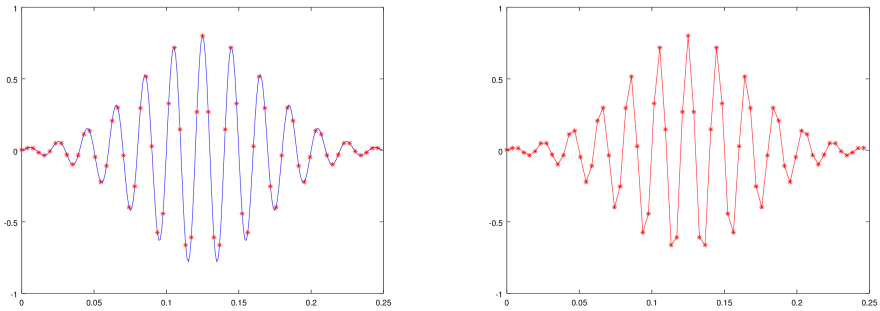


FIGURE 1 – A gauche : le signal original $G$ représenté sur $[0, T]$ en bleu a été échantillonné et on a donc construit le vecteur $u = (u_0, \dots u_{N-1})$ dont les valeurs correspondent aux ordonnées des points rouges. A droite : on trace en les reliant par des segments de droites les points $(t_n, u_n)$ avec $t_n = \frac{n}{f_s}$.

A partir du son numérisé nous pouvons travailler à l'aide d'algèbre linéaire et de mathématiques pour effectuer des transformations sur ce son, et dans les cas qui nous intéressent restaurer des sons qui ont été dégradés.

1. La numérisation n'est en fait que partielle car il faudrait en principe que les valeurs obtenues $(G(nt_s))_{0 \leq n \leq N-1}$ qui sont pour nous des réels soient elles-même discrétisées en ce qu'on appelle des flottants en informatique. Dans ce projet on néglige cet aspect des choses en considérant que les flottants sont les réels

**Question 2: À rédiger, et avec Python**

*Rappel : des routines Python qui peuvent vous être utiles sont disponibles dans la section 4.*

Soit $G : t \mapsto A\sin(2\pi\omega_0 t) + A\sin(2\pi(\omega_0+2)t)$ où $\omega_0$ est un entier et $t$ varie dans $[0,T]$ pour $T = 3$.

1. Déterminer des réels $B$, $\omega_1$ et $\omega_2$ tels que pour tout $t$,
$$G(t) = B\sin(2\pi\omega_1 t)\cos(2\pi\omega_2 t).$$
2. On se donne $f_s = 11025$ dans tout ce qui suit et on pose $t_n = n/f_s$ et $g_n = G(t_n)$ pour $n \leq N-1$ et $N$ tel que $t_{N-1} \leq T < t_N$.
   (a) Calculer $N$ explicitement en fonction de $T$ et $f_s$.
   (b) Tracer avec Python la courbe $(t_n, g_n)_{n=0,\dots,N-1}$ pour $\omega_0 = 440$.
   (c) On prendra dans ce qui suit successivement $\omega_0 = 220$, $\omega_0 = 440$, $\omega_0 = 660$ et $A = 0.8$. Pour chaque valeur de $\omega_0$ et à l'aide de Python, transformer le vecteur $g = (g_n)_{n=0,\dots,N-1}$ en un son. Ecoutez-le à l'aide du lecteur audio de votre choix. Que se passe-t-il lorsque $\omega_0$ augmente ?

## 2.1 Une base bien utile

Nous avons vu que les notes pures se décomposent facilement comme somme de sinusoïdes élementaires. Dans le cas des signaux numériques nous allons pouvoir montrer que tout vecteur de $\mathbb{C}^N$ (et donc tout signal numérique) peut se décomposer comme une combinaison linéaire de sinusoïdes discrétisées.

### 2.1.1 Notations

Nous allons utiliser les outils suivants.

- On fixe $N \geq 2$ un entier naturel.
- On note $\zeta$ le nombre complexe $\zeta = e^{\frac{2i\pi}{N}}$.
- On considère la famille de vecteurs $\mathcal{E} = \{\varepsilon_\ell \in \mathbb{C}^N, \ell = 0,\dots,N-1\}$ tels que pour $\ell = 0,\dots,N-1$ et pour $n \in \{0,\dots N-1\}$ la $n$-ième coordonnée du vecteur $\varepsilon_\ell$ s'écrive
$$(\varepsilon_\ell)_n = \zeta^{ln} = e^{\frac{2i\pi\ell n}{N}}$$
- On notera $\mathcal{C} = \{e_l \in \mathbb{C}^N, l = 0,\dots,N-1\}$ la base canonique de $\mathbb{C}^N$ c'est-à-dire que
$$(e_l)_n = \begin{cases} 0 & \text{si } n \neq l \\ 1 & \text{si } n = l \end{cases}.$$
- On indice les lignes et les colonnes des matrices sur $\mathbb{C}^N$ de 0 à $N-1$.
- La matrice $E$ est la matrice dont les vecteurs colonnes sont les vecteurs de $\mathcal{E}$ écrits dans la base canonique de $\mathbb{C}^N$.
- Pour une matrice $A = (a_{i,j})_{i,j=0..N-1}$ de $\mathcal{M}_N(\mathbb{C})$, on note $A^*$ la matrice de coefficients $\tilde{a}_{i,j} = \overline{a_{j,i}}$.

**Question 3**

1. Montrer que à $\ell$ fixé le vecteur $\varepsilon_\ell$ est la discrétisation sur l'intervalle $[0,2\pi]$ d'une fonction du type $\mathbf{E}_\omega$ dont on précisera $\omega$.
2. Soit $k_0$ fixé un entier naturel non nul et $N > 2k_0$. On prend $f_s = N$. Donner la décomposition à l'aide des vecteurs de $\mathcal{E}$ de $u$, discrétisation d'une note pure de pulsation $2\pi k_0$.

*En réalité nous allons voir que tout son numérisé peut s'écrire à l'aide des vecteurs $\varepsilon_\ell$ car la famille $\mathcal{E}$ forme une base de $\mathbb{C}^N$.*

### 2.1.2 Base de Fourier discrète et changements de base

On a le théorème suivant :

**Théorème 1.** *1. La matrice $E$ est inversible et son inverse est la matrice $\frac{1}{N}E^*$.*

*2. La famille $\mathcal{E}$ est une base de $\mathbb{C}^N$ appelée* base de Fourier discrète.

**Éléments de démonstration.** *Plusieurs démonstration de ce théorème sont possibles :*

- *On peut utiliser le Lemme suivant :*
  **Lemme 1.** *Soit $k_0 \in \mathbb{Z}$ et $N \in \mathbb{N}^\star$. Alors*
  — *si $k_0 \equiv 0(modN)$ on a $\sum_{n=0}^{N-1} \zeta^{k_0 n} = N$,*
  — *sinon $\sum_{n=0}^{N-1} \zeta^{k_0 n} = 0$.*
  *et l'utiliser pour calculer directement le produit $E^*E$*
- *On peut remarquer que $E$ est également la matrice de l'application linéaire*

$$\begin{array}{rcl} \mathcal{L} : \mathbb{C}_{N-1}[X] & \to & \mathbb{C}^N \\ P & \mapsto & (P(1), P(\zeta), \ldots, P(\zeta^{N-1})) \end{array}$$

*entre la base canonique $\{1, X, \ldots, X^{N-1}\}$ de l'espace $\mathbb{C}_{N-1}[X]$ des polynômes à coefficients complexes de degré inférieur ou égal à $N-1$ et la base canonique $\mathcal{C}$ de $\mathbb{C}^N$.*

**Question 4: À rédiger**

1. Dans le cas particulier $N = 4$ démontrer le théorème 1.
2. Démontrer le Lemme 1 et le théorème par la première méthode.
3. Démontrer le théorème à l'aide de la deuxième méthode, en suivant les étapes suivantes :
   (a) Montrer que l'application linéaire $\mathcal{L}$ est injective puis bijective.

   *Nous allons maintenant calculer $\mathcal{L}^{-1}$ puis sa matrice dans les bases qui nous intéressent.*

   (b) On pose pour $k = 0, 1, \ldots, N-1$, $\widetilde{L}_k(X) = \prod_{j \neq k}(X - \zeta^j)$. Montrer que $\widetilde{L}_k(\zeta^k) \neq 0$.
   (c) On pose $L_k(X) = \frac{1}{\widetilde{L}_k(\zeta^k)}\widetilde{L}_k(X)$. Montrer que $\mathcal{L}^{-1}(v_0, \ldots, v_{N-1}) = \sum_{k=0}^{N-1} v_k L_k(X)$. Il s'agit du polynôme d'interpolation de Lagrange aux points $(1, \zeta, \ldots, \zeta^{N-1})$ et aux valeurs $(v_0, v_1, \ldots, v_{N-1})$.
   (d) Montrer que $\prod_{j=0}^{N-1}(X - \zeta^j) = X^N - 1$. En déduire que

$$\zeta^k \widetilde{L}_k(X) = \frac{\left(\frac{X}{\zeta^k}\right)^N - 1}{\frac{X}{\zeta^k} - 1} = \sum_{p=0}^{N-1} \left(\frac{X}{\zeta^k}\right)^p.$$

   (e) En déduire que $\widetilde{L}_k(\zeta^k) = N\zeta^{-k}$ puis que

$$L_k(X) = \frac{1}{N} \sum_{p=0}^{N-1} \frac{1}{\zeta^{kp}} X^p.$$

   Conclure sur les coefficients de $E^{-1}$ et montrer que $E^{-1} = \frac{1}{N}E^*$.

Maintenant qu'on sait que $\mathcal{E}$ est une base de $\mathbb{C}^N$, et si $x$ un vecteur de de $\mathbb{C}^N$ on notera

- $\frac{1}{N}(\hat{x}_0, \hat{x}_1, \ldots, \hat{x}_{N-1})^T$ le vecteur colonne des coordonnées du vecteur $x$ dans la base $\mathcal{E}$. On les appelle les $\hat{x}_i$ pour $i = 0, \ldots, N-1$ les coefficients de Fourier discrets.
- $\hat{x}$ le vecteur colonne $(\hat{x}_0, \hat{x}_1, \ldots, \hat{x}_{N-1})^T$.

- et toujours $(x_0, x_1, \ldots, x_{N-1})^T$ le vecteur colonne des coordonnées de $x$ dans la base canonique.

**Question 5: À rédiger**

1. Écrire les relations de passage de $(x_0, \ldots, x_{N-1})^T$ à $(\hat{x}_0, \ldots, \hat{x}_{N-1})^T$ dans les deux sens.
2. Soit $x \in \mathbb{R}^N$.
   (a) Montrer que les coordonnées de $x$ dans la base de Fourier discrète vérifient $\hat{x}_0 \in \mathbb{R}$ et pour tout $l = 1, 2, \ldots, N-1$, $\overline{\hat{x}_l} = \hat{x}_{N-l}$.
   (b) Réciproquement, montrer que si $y \in \mathbb{C}^N$ vérifie $y_0 \in \mathbb{R}$ et pour tout $l = 1, 2 \ldots, N-1$, $\overline{y_l} = y_{N-l}$ alors il existe $x \in \mathbb{R}^N$ tel que $y = \hat{x}$.

## 2.2 En pratique

Le calcul de $\hat{x}$ n'est pas effectué tel quel par un ordinateur.

En effet un calcul direct donnerait en réalité pour chaque coordonnée de $\hat{x}$, $N$ multiplications suivies de $N-1$ additions, soit un coût d'environ $2N^2$ opérations pour le calcul de $\hat{x}$ (*Pourquoi ?*). Dès que $N$ commence à être grand (par exemple ici $N = 10^6$, ce qui est une taille très courante en image ou même en son), le calcul direct devient très coûteux, voire impossible à mener.

Heureusement dans le cas où on a $N = 2^{n_0}$ avec $n_0 \in \mathbb{N}^*$, nous avons la possibilité d'utiliser un algorithme basé sur des factorisations matricielles astucieuses pour effectuer le calcul de $\hat{x}$ à partir de $x$. Cet algorithme est appelé « Transformée de Fourier rapide » ou encore FFT en anglais (Fast Fourier Transform) et c'est un des algorithmes les plus efficaces du monde. Il permet de calculer $\hat{x}$ en $\mathcal{O}(N \ln(N))$ opérations, c'est à dire que le nombre d'opérations pour effectuer le calcul est proportionnel à $N \ln(N)$.

Un algorithme similaire nous permet de calculer $x$ à partir de $\hat{x}$, il a pour nom en anglais IFFT (*Inverse Fast Fourier Transform*).

**Question 6: À rédiger**

Un échantillonnage d'un morceau de 3mn de qualité standard donnera environ $N = 8 \cdot 10^6$ ($T = 180s$ avec un échantillonnage à $f_s = 44100Hz$ ou $s^{-1}$). Un ordinateur standard de notre époque peut effectuer environ $200 \cdot 10^9$ opérations par secondes (200 gigaFlops). Combien de temps environ prendra le calcul des coefficients de Fourier discrets par la méthode naïve et par la transformée de Fourier rapide ?

Soit un vecteur $x$ de $\mathbb{R}^N$ qui représente la numérisation d'un son échantillonné à la fréquence $f_s = N/T$. Lorsque $0 \leq l \leq N/2$, on dit que $|\hat{x}_l|$ représente *l'intensité de la fréquence* $l f_s/N$ dans le son. Si $N/2 < l \leq N-1$, on dit que $|\hat{x}_l|$ représente *l'intensité de la fréquence* $-(N-l)f_s/N$ (ainsi, $|\hat{x}_{N-1}|$ est l'intensité de la fréquence $-f_s/N$).

**Question 7: Avec Python**

1. On prend $N = f_s = 4096$ et $T = 1$. Pour $\ell = 400, 600, 1000$ transformer les parties réelles des vecteurs $\varepsilon_\ell$ notées $\mathcal{R}e(\varepsilon_\ell)$ en son. Ecouter les. Que se passe-t-il quand $\ell$ augmente ?
2. Combien vaut $\widehat{\mathcal{R}e(\varepsilon_\ell)}$ ? Retrouver le résultat numériquement.

**Question 8: Avec Python**

On cherche à connaître les deux notes fondamentales jouées par le saxophoniste dans l'extrait `2notes.wav`.

*Un son harmonique $s$ est composé d'un ensemble de $N$ sinusoïdes de fréquences $\omega_n = n\omega_0$, pour $n = 1, \ldots, N$. Ces sons purs sont appelés les harmoniques du son $s$, $n$ est appeleé le rang de l'harmonique, et $\omega_0$ est la fréquence fondamentale (harmonique de rang 1). On a donc pour un son harmonique une formule du type, (avec $A_n$ l'amplitude de chaque harmonique),*

$$s(t) = \sum_{n=1}^{K} A_n cos(n\omega_0 t)$$

*Les instruments de musique produisent des notes qui sont en fait des sons harmoniques. Ainsi en première approximation un instrument de musique produit un son du type*

$$f(t) = B(t)\left(\sum_{n=1}^{K} A_n cos(n\omega_0 t)\right)$$

*où $B$ est une fonction très régulière qui varie lentement.*

1. Écouter le son `2notes.wav` et visualiser le vecteur $u = (u_n)_{n=0,\ldots N-1}$ correspondant à ce fichier en traçant $u_n$ en fonction de $n$.
2. Tracer le module des coordonnées $\hat{u}_n, n = 0, \ldots N-1$ de $u$ et déterminer les fréquences fondamentales des deux notes jouées dans le morceau.

# 3 Restauration d'un son basse fréquence

Notre technique se base sur l'hypothèse (le modèle) suivant : il est possible de trouver facilement deux sous-espaces vectoriels de $\mathbb{R}^N$ en somme directe $U$ et $V$ tels que $u \in U$ et $g \in V$, et on peut récupérer $u$ à partir de $y$ en effectuant tout simplement la projection de $y$ sur $U$ parallèlement à $V$.

Cette vision très générale n'est cependant pas encore opérationnelle car il faut maintenant préciser quels peuvent être les espaces $U$ et $V$ et il faut également que la projection sur $U$ parallèlement à $V$ puisse être effectuée efficacement. Enfin tout ceci doit pouvoir être mis en œuvre algorithmiquement sur un ordinateur.

Une idée classique en traitement du son est de voir si les sous-espaces $U$ et $V$ peuvent être des sous-espaces vectoriels engendrés par certains vecteurs donnés de la *base de Fourier discrète*.

La théorie de Fourier que vous apprendrez plus tard nous apprend que la répartition des fréquences dans une séquence sonore caractérise les propriétés musicales du son (notes, accords, timbres des instruments...). Inversement, quand le son étudié est assez simple, on peut avoir une bonne idée *a priori* de l'ensemble des fréquences qui vont apparaître dans le son. Si on connaît l'origine des perturbations on peut également avoir une idée des fréquences qui apparaissent à cause des perturbations.

Nous faisons ici l'hypothèse que le son originel est basse fréquence alors que la dégradation est haute fréquence. On suppose donc qu'il existe un entier $k_0$ tel que $u$ est dans l'espace

$$U_{k_0} = \mathrm{Vect}(\varepsilon_0, \ldots, \varepsilon_{k_0}, \varepsilon_{N-k_0}, \ldots, \varepsilon_{N-1})$$

alors que $g$ est dans l'espace

$$V_{k_0} = \mathrm{Vect}(\varepsilon_{k_0+1}, \ldots, \varepsilon_{N-k_0-1}).$$

Il est donc en principe très simple de retrouver $u$ à partir de $y$ : il suffit de calculer $\hat{y}$, de mettre à 0 les hautes fréquences puis de reconstruire le son à partir des fréquences gardées (c'est-à-dire calculer les coordonnées dans la base canonique de $\mathbb{R}^N$ à partir des coordonnées de Fourier discrets).

**Question 9: Avec Python**

1. Commenter la méthode proposée.
2. En réécoutant les sons indiquer qui selon vous des deux sons originaux $u^0$ et $u^1$ pourrait vérifier cette hypothèse. Calculer $\widehat{u^0}$ et $\widehat{u^1}$. Visualiser le module de ses coordonnées et confirmer si l'un des deux sons $u^0$ ou $u^1$ appartient à un sous-espace vectoriel du type $U_{k_0} = Vect(\varepsilon_0, \ldots, \varepsilon_{k_0}, \varepsilon_{N-k_0}, \ldots, \varepsilon_{N-1})$ engendré par des vecteurs de $\mathcal{E}$.
3. Restaurer ce son $u^i$ à partir de $y^i$. On expliquera pourquoi la fonction `filtrage` de la partie 4 ci-dessous permet de réaliser cette opération et on vérifiera en pratique que la perturbation a bien été éliminée.

Attention : la stratégie de débruitage proposée dans ce texte repose de façon essentielle sur la structure du son originel ainsi que celle de la pertubation. Elle ne marchera pas dans tous les cas !

**Question 10**

Justifier l'affirmation précédente à l'aide de raisonnements théoriques ou d'illustrations numériques. On pourra par exemple utiliser le second son pour appuyer l'argumentation.

# 4 Routines Python pour résoudre ces problèmes

- importation d'un fichier `musique.wav` en un vecteur `y` (en conservant l'information sur la fréquence d'échantillonnage `fs`)

```
import scipy.io.wavfile as wavfile

fs, y = wavfile.read('musique.wav')
```

- exportation d'un vecteur `y` en un fichier `musique.wav` (à l'aide de l'information sur la fréquence d'échantillonnage `fs`)

```
import numpy as np
import scipy.io.wavfile as wavfile

wavfile.write('musique.wav',fs,y)
```

On désigne par `ychap` le vecteur $\hat{y}$.

- calcul de `ychap` à partir du vecteur `y`

```
import numpy.fft as npft

ychap = npft.fft(y) # les valeurs de ychap sont complexes.
```

- calcul de `y` à partir du vecteur `ychap`

```
import numpy.fft as npft

y = npft.ifft(ychap)

# les valeurs de y sont complexes en general apres cette operation dans tous
    les cas, a cause par exemple des erreurs d'arrondi.

# pour forcer le cas echeant y a etre reel (pour reconstruire un son numerique)
     on ajoute
y = y.real
```

- Fonction qui calcule la projection d'un vecteur `y` sur un sous-espace du type $U_{k_0}$.

```
def filtrage(M,y):
    ychap = np.fft.fft(y)
    N = ychap.size
    mask = np.ones(N)
    coord = np.arange(M+1,N-M,1)
    mask[coord]=0
    yfiltchap = mask*ychap
    yfilt = np.fft.ifft(yfiltchap)
    yfilt = yfilt.real
    return yfilt
```